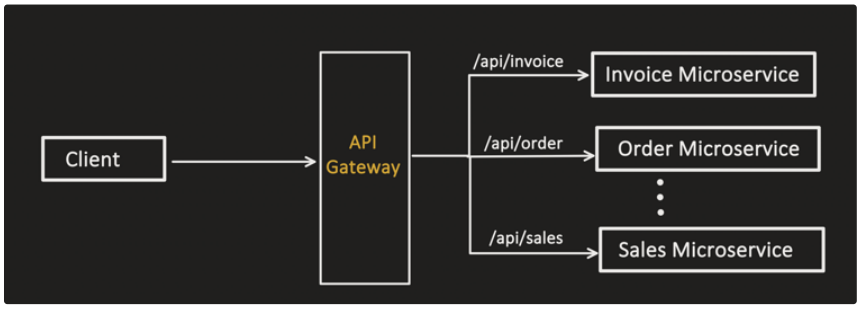
API Gateway - Part2

API Gateway:

It provides a Single entry point to access all microservices.



And this Single Entry Point, provides lot of benefits like:

1. Routing : forward requests to right microservices.
2. Load balancing
3. Authentication like JWT
4. Rate Limiting
5. Resilience features (Circuit breaker, Retry etc.)
6. Request/Response Transformation
7. Monitoring and Logging.

Etc.

Concept && Coding

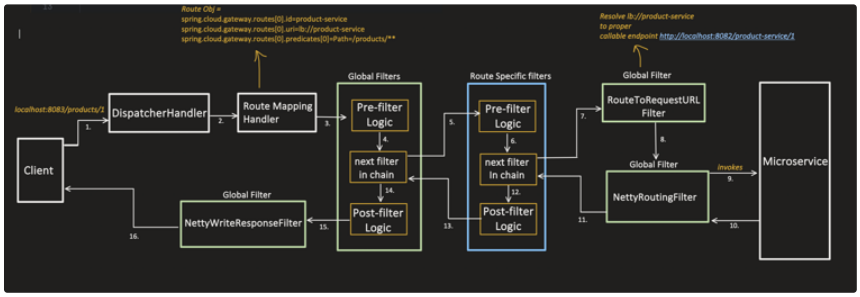
Now to apply Authentication, Rate Limiting, Resilience etc. in API Gateway, we need to understand:

FILTERS in API Gateway:

It can intercept and modify the HTTP request and response as they passes through the gateway.

```
spring.application.name=apigateway
server.port=8083

# Route: Forward /products/** to product-service
```

```
spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=http://localhost:8082
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

# Route: Forward /orders/** to order-service
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=http://localhost:8081
spring.cloud.gateway.routes[1].predicates[0]=Path=/orders/**

```



One Question should have come to your mind**: "why some Global filters are after the Route Specific Filters"**

Answer is**: ORDERING**

In API Gateway, each filter (either Global or Route-Specific) is given an ORDER. Concept && Coding

Minimum Order Value = high priority

Maximum Order Value = Low priority

ex: filter with order value -1 will run before filter with order value 0.

So Generally, most of the Global filters are given lower value than Route-Specific filters but some Global filters like RouteToRequestUrlFilter and NettyRoutingFilter intentionally given higher value, so that they execute last:

- **RouteToRequestUrlFilter**

```
public static final int ROUTE_TO_URL_FILTER_ORDER = 10000;

```

NettyRoutingFilter

```
public static final int ORDER = Ordered.LOWEST_PRECEDENCE;

```

By Default values:

For **Global filter**: if we don't define the Order, by-default value = "Ordered.LOWEST_PRECEDENCE"

For **Route-Specific filter**: say if we have 3 route specific filters then by-default index value (0,1,2) is assigned as Order value.

Global Filters:

We have already seen, so many pre-existing Global Filter in above flow.

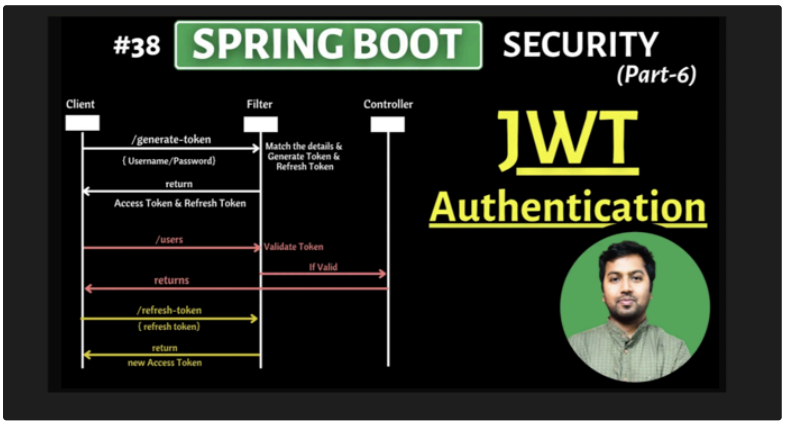
But lets create our own custom Global Filter for doing "**Authentication**".

```java
@Component
public class JwtAuthGlobalFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {

        String uriPath = exchange.getRequest().getURI().getPath();

        // Skip JWT validation for auth endpoints
        if (uriPath.startsWith("/auth")) {
            return chain.filter(exchange);
        }

        // Extract Authorization header
        String authHeader = exchange.getRequest()
                .getHeaders()
                .getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null) {
            // if token is missing

exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        try {
            // verify JWT token logic here.....
        } catch (Exception e) {

exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // call next filter chain and also add post-filter logic
        return chain.filter(exchange).then(
                Mono.fromRunnable(() -> {
                    // post filter logic here
                })
        );
    }

    @Override
    public int getOrder() {
        // lower value means higher priority
        return -1;
    }
```

Concept && Coding

```
}

```

If there is any doubt, what is JWT and how to verify the JWT, kindly check the JWT implementation video:



Route-Specific Filters:

There are many pre-existing Route-specific filters available in Spring Cloud Gateway like:

- **AddRequestHeader** : used to add a header to the request.
- **AddResponseHeader** : used to add a header to the response.
- **RemoveRequestHeader** : used to remove a request header.
- **RemoveResponseHeader** : used to remove a response header.
- **Retry** : used to retry the request on failure (configurable)
- **RequestRateLimiter** : used to apply rate limiting
- **CircuitBreaker** : used to stop making repeated call to failed downstream.

Etc...

Concept && Coding

Let's first see, how to use pre-existing Route-specific filters, then will write our own custom filter.

```
spring.application.name=apigateway
server.port=8083

# Route: Forward /products/** to product-service
spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

## FILTERS ##

# Remove a request header
spring.cloud.gateway.routes[0].filters[0].name=RemoveRequestHeader
spring.cloud.gateway.routes[0].filters[0].args.name=Api-private-Key

```

```
# path of the Eureka Server to fetch the microservices host details
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Lets see an example with multiple route-specific filters:

```
spring.application.name=apigateway
server.port=8083

spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

## FILTERS ##
# Add a request header
spring.cloud.gateway.routes[0].filters[0].name=AddRequestHeader
spring.cloud.gateway.routes[0].filters[0].args.name=X-Test-RequestHeader
spring.cloud.gateway.routes[0].filters[0].args.value=ApiGatewayRequest

# Add a response header
spring.cloud.gateway.routes[0].filters[1].name=AddResponseHeader
spring.cloud.gateway.routes[0].filters[1].args.name=X-Test-ResponseHeader
spring.cloud.gateway.routes[0].filters[1].args.value=ApiGatewayRespons
e

# Remove a request header
spring.cloud.gateway.routes[0].filters[2].name=RemoveRequestHeader
spring.cloud.gateway.routes[0].filters[2].args.name=Api-private-Key

```

Concept && Coding

```
# path of the Eureka Server to fetch the microservices host details
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Now we know, in What would be the class name and Config going to looks like:

Java class name is**: *AddRequestHeaderGatewayFilterFactory.java***
Config is:

Public static class NameValueConfig {

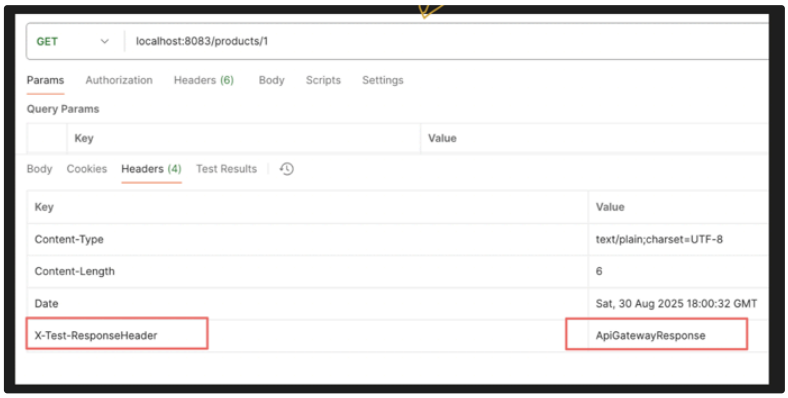
Protected String name;

Protected String value;

.

.

}



Lets see an RETRY route-specific filter:

```
spring.application.name=apigateway
server.port=8083

spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

## FILTERS ##
# retry on failure, max 4 attempts, on GET and DELETE requests only
spring.cloud.gateway.routes[0].filters[0].name=Retry
spring.cloud.gateway.routes[0].filters[0].args.retries=4
spring.cloud.gateway.routes[0].filters[0].args.methods=GET,DELETE

# path of the Eureka Server to fetch the microservices host details
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Concept && Coding

Now we know, in What would be the class name and Config going to looks like:

Java class name is**: *RetryGatewayFilterFactory.java***
Config is:

Public static class RetryConfig {

int retries

List<HttpStatus> statuses

List<HttpMethod> methods

.

.

.

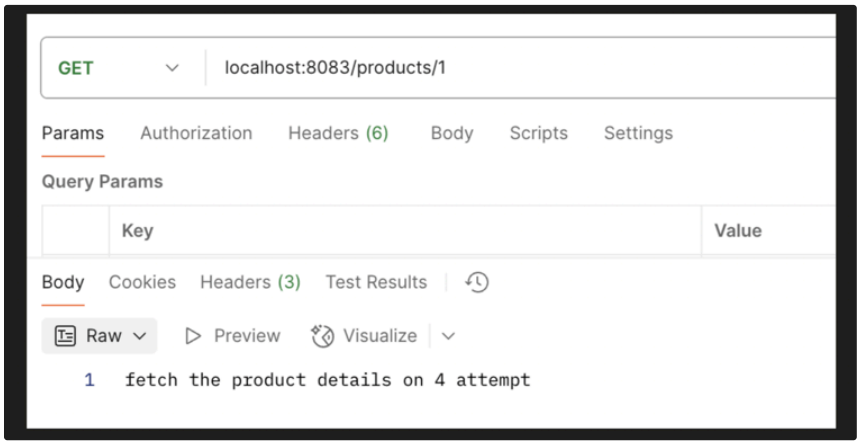
}

Product Microservice Controller:

Just to test, if APIGateway is properly applying the RETRY filter or not, purposefully failing the endpoint for 3 times and from 4th call onwards it will pass.

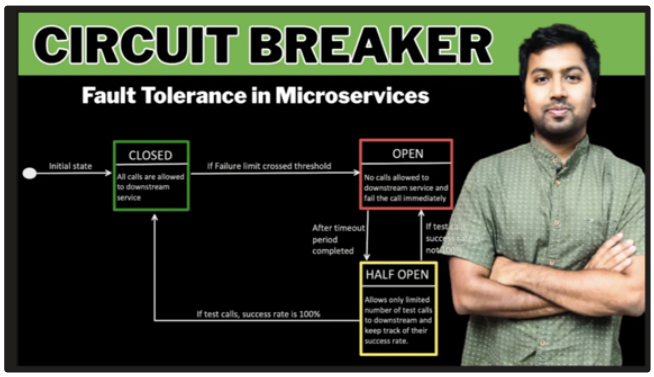
```java
@RestController
@RequestMapping("/products")
public class ProductController {

    int counter = 0;

    @GetMapping("/{id}")
    public ResponseEntity<String> getProduct(@PathVariable String id) {

        counter++;

        if (counter <= 3) {
            return ResponseEntity
                    .status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("Fail");
        }

        return ResponseEntity
                .ok()
                .body("fetch the product details on " + counter + " attempt");
    }
}

```

Concept && Coding



Lets see Circuit Breaker route-specific filter:

If you have any doubt with CircuitBreaker topic, kindly check out the detailed video in this series:



pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>
</dependency>

```

Concept && Coding

application.properties

```
spring.application.name=apigateway
server.port=8083

spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

## FILTERS ##

# Circuit breaker (redirect to fallback endpoint if service fails)
spring.cloud.gateway.routes[0].filters[0].name=CircuitBreaker
spring.cloud.gateway.routes[0].filters[0].args.name=myCircuitBreakerName
spring.cloud.gateway.routes[0].filters[0].args.fallbackUri=forward:/fallback

# CircuitBreaker config
resilience4j.circuitbreaker.instances.myCircuitBreakerName.sliding-window-type=COUNT_BASED
resilience4j.circuitbreaker.instances.myCircuitBreakerName.sliding-window-size=10
resilience4j.circuitbreaker.instances.myCircuitBreakerName.minimum-number-of-calls=5
resilience4j.circuitbreaker.instances.myCircuitBreakerName.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.myCircuitBreakerName.wait-duration-in-open-state=10s
```

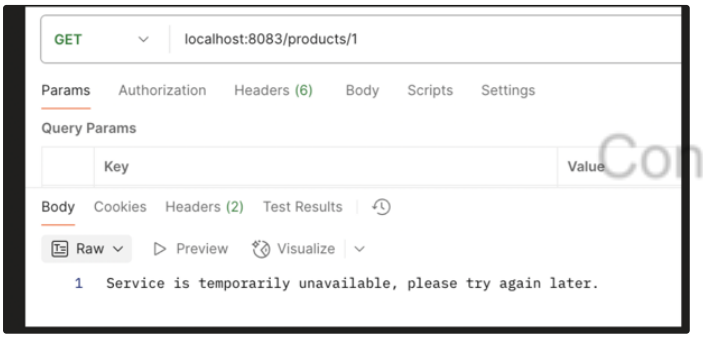
```
resilience4j.circuitbreaker.instances.myCircuitBreakerName.permitted-number-of-calls-in-half-open-state=3
logging.level.io.github.resilience4j.circuitbreaker=DEBUG

# path of the Eureka Server to fetch the microservices host details
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Controller

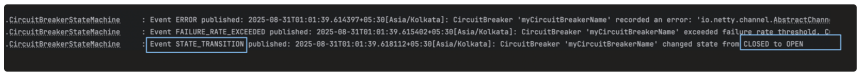
```
@Controller
public class FallbackController {

    @GetMapping("/fallback")
    public ResponseEntity<String> fallback() {
        return ResponseEntity.ok("Service is temporarily unavailable, please try again later.");
    }
}

```

For testing I did not started the Product Microservice server:

Called multiple times, more than 5 times, failure threshold got crossed:



Concept && Coding



Lets write one Custom Route-specific filter:

pom.xml

```
# Custom Route Specific filter testing
spring.cloud.gateway.routes[0].filters[5].name=CustomRoute
spring.cloud.gateway.routes[0].filters[5].args.country=India

```

```
@Component
public class CustomRouteGatewayFilterFactory
        extends AbstractGatewayFilterFactory<CustomRouteGatewayFilterFactory.CustomConfig> {

    public CustomRouteGatewayFilterFactory() {
        super(CustomConfig.class);
```

```java
    }

    @Override
    public GatewayFilter apply(CustomConfig config) {
        return (exchange, chain) -> {

            System.out.println("pre filter logic here, config value: " + config.getCountry());

            return chain.filter(exchange).then(Mono.fromRunnable(() -> {
                System.out.println("post filter logic here");
            }));
        };
    }

    public static class CustomConfig {
        private String country;

        public String getCountry() {
            return country;
        }

        public void setCountry(String country) {
            this.country = country;
        }
    }
}

```

```
2025-08-31T02:11:29.146+05:30  INFO 25700 --- [apigateway] [foReplicator-%d] com.netflix.discovery.DiscoveryClient    : DiscoveryClient_APIGATEWAY/192.168
2025-08-31T02:11:29.199+05:30  INFO 25700 --- [apigateway] [           main] o.s.b.web.embedded.netty.NettyWebServer  : Netty started on port 8083 (http)
2025-08-31T02:11:29.199+05:30  INFO 25700 --- [apigateway] [           main] .s.c.n.e.s.EurekaAutoServiceRegistration : Updating port to 8083
2025-08-31T02:11:29.217+05:30  INFO 25700 --- [apigateway] [           main] c.c.apigateway.ApigatewayApplication     : Started ApigatewayApplication in 1
pre filter logic here, config value: India
2025-08-31T02:11:34.442+05:30 ERROR 25700 --- [apigateway] [ctor-http-nio-3] i.n.r.d.DnsServerAddressStreamProviders  : Unable to load io.netty.resolver.d
post filter logic here
2025-08-31T02:11:34.474+05:30 DEBUG 25700 --- [apigateway] [ctor-http-nio-3] i.g.r.c.i.CircuitBreakerStateMachine     : CircuitBreaker 'myCircuitBreakerNa
2025-08-31T02:11:34.475+05:30 DEBUG 25700 --- [apigateway] [ctor-http-nio-3] i.g.r.c.i.CircuitBreakerStateMachine     : Event SUCCESS published: 2025-08-3
```

Concept && Coding